Doctor Appointment System

A complete, end-to-end explanation of the project:
every feature, every file, and every technology it uses

React + Vite + Tailwind (frontend) | Node.js + Express + Sequelize (backend) |
PostgreSQL (database) | JWT (login sessions)

How to read this document: Part 1 explains what the app is and the basic ideas behind it, in plain words. Part 2 explains the technologies (PostgreSQL, JWT, bcrypt, Sequelize and the rest) from scratch. Part 3 follows real actions through the system from start to finish. Parts 4 and 5 go through every file in the project, one by one. Part 6 lists every API endpoint, every business rule and the scripts. Part 7 is a glossary of every term used.

Contents

Part 1 - What this project is **4**

1.1 The one-paragraph summary . . . 4
1.2 The three kinds of users (roles) . . . 4
1.3 The big picture: three programs working together . . . 4
1.4 The life of an appointment in plain words . . . 4
1.5 Appointment statuses and slot statuses . . . 5

Part 2 - The technologies, explained from zero **6**

2.1 JavaScript, Node.js and npm . . . 6
2.2 HTTP, REST APIs and JSON . . . 6
2.3 PostgreSQL - the database . . . 6
2.4 Sequelize - talking to Postgres from JavaScript (ORM) . . . 8
2.5 Authentication: passwords, bcrypt and JWT . . . 8
2.6 Express and middleware . . . 9
2.7 Zod - input validation . . . 10
2.8 File uploads - multer and safety checks . . . 10
2.9 Background jobs - node-cron . . . 10
2.10 E-mail - nodemailer . . . 10
2.11 dotenv and the .env file . . . 11
2.12 The frontend stack . . . 11

Part 3 - End-to-end flows (what happens, step by step) **12**

3.1 Starting the app (start.bat) . . . 12
3.2 Opening the website . . . 12
3.3 Patient registration and login . . . 12
3.4 Doctor registration and approval . . . 12
3.5 Doctor publishes availability . . . 13
3.6 Patient searches and books . . . 13
3.7 Confirm, "I can't make it", cancel . . . 13
3.8 The every-minute job . . . 13
3.9 The visit, completion and prescription . . . 14
3.10 Notifications and badges . . . 14
3.11 Admin: users and settings . . . 14

Part 4 - Every file: project root and server **15**

4.1 Project root . . . 15
4.2 server/ (configuration files) . . . 15
4.3 server/src - entry points . . . 15
4.4 server/src/config . . . 15
4.5 server/src/models . . . 16
4.6 server/src/migrations . . . 17
4.7 server/src/middlewares . . . 17

4.8 server/src/validators . . . 17
4.9 server/src/utils . . . 17
4.10 server/src/services (business logic) . . . 18
4.11 server/src/controllers (thin HTTP handlers) . . . 19
4.12 server/src/routes and jobs . . . 19
4.13 server/src/scripts . . . 19
Part 5 - Every file: client (frontend) 20
5.1 client/ (configuration files) . . . 20
5.2 client/src - entry and routing . . . 20
5.3 client/src/context . . . 21
5.4 client/src/lib (shared helpers) . . . 21
5.5 client/src/components . . . 22
5.6 client/src/pages . . . 22
Part 6 - Reference: API, rules, scripts, security 24
6.1 Every API endpoint (all under /api) . . . 24
6.2 Business rules summary . . . 25
6.3 Security checklist . . . 25
6.4 Commands you will use . . . 25
6.5 Demo accounts . . . 26
6.6 Common problems . . . 26
Part 7 - Glossary 27

Part 1 - What this project is

1.1 The one-paragraph summary

This project is a **doctor appointment booking website**. Patients find a doctor, pick a free time slot and book it. Doctors register, prove they are real doctors by uploading a certificate, publish the times they are free, see their bookings, finish visits and write prescriptions. An administrator approves doctors, manages users, sets the platform commission (the platform's cut) and changes the app's name and logo. The system sends reminders, asks patients to confirm they will attend, and automatically moves bookings that are not confirmed in time.

1.2 The three kinds of users (roles)

Role	Internal name	What they can do
Patient	user	Register, search doctors, see fees, book a slot, confirm attendance, "I can't make it" (move to the next free slot), cancel, read prescriptions, receive notifications, see a dashboard.
Doctor	doctor	Register with a certificate, wait for approval (re-upload if rejected), edit profile and fee, create availability slots in bulk, delete/reopen slots, see appointments, mark completed or no-show, write prescriptions, see today's schedule and earnings.
Admin	admin	Review doctors (preview certificate, approve, reject, revoke), activate/deactivate any user, see every appointment, set app name/tagline/logo, commission (percentage or flat) and currency, see platform statistics and commission earned.

Note: in the code a patient is called `user`. This is just a name; on screen it says "Patient".

1.3 The big picture: three programs working together

When the app runs, three separate programs are running at once:

- **The frontend (client)** - a React app that runs *inside your web browser*. It draws the pages, buttons and forms. In development it is served by **Vite** at `http://localhost:5173`.
- **The backend (server / API)** - a Node.js program using **Express**. It runs on your computer at `http://localhost:5000/api`. It holds all the rules (who may do what, fees, the 24-hour lock...) and is the only part that talks to the database.
- **The database** - **PostgreSQL**, a separate server program listening on port 5432. It stores everything permanently: users, doctors, slots, appointments, prescriptions, notifications, reminders and settings.

```
Browser (React app, port 5173)
      |   HTTP requests: GET /api/doctors, POST /api/appointments ...
      |   (Vite forwards /api/* to port 5000 in development)
      v
Express API (Node.js, port 5000)  --- checks the JWT, validates input, applies the rules
      |   SQL queries (written for us by Sequelize)
      v
PostgreSQL database (port 5432)  --- tables: users, doctor_profiles, slots, appointments, ...

+ a background job inside the API that runs every minute (reminders, unconfirmed bookings)
```

The browser **never** talks to the database directly. It only asks the API, and the API decides what is allowed. That is the most important security idea in the whole project: **the server is the authority**. The frontend repeats some checks (for example password rules) only so users see mistakes sooner.

1.4 The life of an appointment in plain words

- A doctor publishes free slots, for example Mon-Fri, 10:00-13:00, 30 minutes each.
- A patient books a slot. If the visit is **more than 24 hours away**, the booking starts as *Awaiting confirmation*. The patient must press "Confirm" before *start time minus 24 h*.
- If the visit is **less than 24 hours away**, it is confirmed at once, but it can no longer be cancelled or moved.
- Reminders are sent 1 day, 1 hour and 15 minutes before (only the ones still in the future), plus a "please confirm" nudge.

- If the patient never confirms, a background job **moves the booking to the doctor's next free slot** at least 48 h ahead and asks again. After 2 automatic moves, or if no slot is left, it is cancelled.
- At the visit time the doctor marks it *Completed* (or *No-show*) and can write a prescription. The patient is notified.
- The price (doctor fee + platform commission) is **copied onto the appointment at booking time**, so later price changes never affect it.

1.5 Appointment statuses and slot statuses

Appointment status	Meaning
pending_confirmation	Booked, but the patient still has to confirm. Shown as "Awaiting confirmation".
confirmed	The patient will attend (or booked inside 24 h, or moved the time themselves).
completed	The doctor finished the visit.
cancelled	Cancelled by the patient, doctor, admin or the system. cancelled_by records who.
no_show	The patient did not come.

Slot status	Meaning
available	Free; patients can book it.
booked	Holds an active appointment.
blocked	The doctor cancelled an appointment here because they are unavailable; it is not reopened automatically. The doctor can "unblock" it.

Doctor status	Meaning
pending	Registered, waiting for the admin to check the certificate. Can sign in, but cannot publish slots or be found by patients.
approved	Verified. Appears in search and can take bookings.
rejected	Not approved (a reason is given). Can upload a new certificate, which puts them back to pending.

Part 2 - The technologies, explained from zero

2.1 JavaScript, Node.js and npm

JavaScript is the programming language of the whole project, both frontend and backend. **Node.js** lets JavaScript run outside a browser, as a normal program; the backend is a Node.js program. **npm** (Node Package Manager) downloads libraries other people wrote. Each half of the project has a `package.json` that lists the libraries it needs and short commands ("scripts") like `npm run dev`. `npm install` downloads them into a `node_modules` folder (never committed to git). `package-lock.json` records the exact versions installed, so every machine gets the same ones.

The server uses **CommonJS** modules (`require(...)` / `module.exports`); the client uses **ES modules** (`import` / `export`), set by `"type": "module"` in its package.json.

2.2 HTTP, REST APIs and JSON

The browser and the server talk over **HTTP**. Each message is a **request** with a **method** and a **path**, and the server sends back a **response** with a **status code** and a body. This project uses **JSON** (JavaScript Object Notation, text like `{"name":"Priya"}`) for request and response bodies, except file uploads, which use **multipart/form-data**.

Method	Used for	Example in this project
GET	Read data	GET /api/doctors?limit=6
POST	Create something or perform an action	POST /api/appointments, POST /api/appointments/:id/confirm
PATCH	Change part of something	PATCH /api/auth/me, PATCH /api/admin/doctors/:id/status
PUT	Replace/save a whole thing	PUT /api/appointments/:id/prescription, PUT /api/admin/settings
DELETE	Remove something	DELETE /api/doctor/slots/:id

Status	Meaning here
200 / 201	OK / Created
400	Bad input (VALIDATION_ERROR, INVALID_FILE_TYPE, REASON_REQUIRED...)
401	Not signed in, or the token is invalid/expired, or the account is deactivated
403	Signed in but not allowed (wrong role, doctor not approved yet)
404	Not found - also used when something exists but is not yours, so ids cannot be probed
409	Conflict (slot just taken, duplicate e-mail, already approved)
422	Request is valid but a business rule refuses it (LOCK_WINDOW, NOT_STARTED)
429	Too many attempts (rate limit on login/register)
500	Unexpected server error; details stay in the server log

Every response from this API has the same shape. Success: `{ "success": true, "data": ..., "meta": {...} }`. Error: `{ "success": false, "code": "LOCK_WINDOW", "message": "...", "details": ... }`. The *code* is a stable machine-readable name the frontend can react to; the *message* is safe to show to the user.

2.3 PostgreSQL - the database

PostgreSQL ("Postgres") is a **relational database**: data lives in **tables** (like spreadsheets) with fixed **columns**, and each record is a **row**. You talk to it in **SQL** (Structured Query Language), for example `SELECT * FROM users WHERE email = 'a@b.c'`. It runs as its own server program (port 5432). On this machine it is a "portable" copy in `C:\Users\Me\pgsql`; its data files live in `pgsql\data`, created once with `initdb`, started and stopped with `pg_ctl`.

Core database ideas used in this project

- **Primary key (PK)** - the unique id of each row. Here every table uses a **UUID**, a random 128-bit id like `da9771c4-1a19-4112-...`, made by Postgres's `gen_random_uuid()`. UUIDs cannot be guessed or counted, unlike 1, 2, 3.
- **Foreign key (FK)** - a column pointing to another table's PK, e.g. `appointments.user_id -> users.id`. Postgres refuses ids that do not exist. **ON DELETE CASCADE** deletes child rows with the parent; **SET NULL** just empties the link.
- **Index** - a lookup structure that makes searches on a column fast (like a book index). A **unique index** also forbids duplicates.
- **Partial unique index** - unique only for rows matching a condition. Used here as the final guard against double booking (see 2.3.2).
- **CHECK constraint** - a rule the database enforces on every row, e.g. `end_at > start_at`, `fee >= 0`, settings `id = 1`.
- **ENUM type** - a column that only accepts a fixed list of words, e.g. role in (user, doctor, admin).
- **timestamptz** - "timestamp with time zone". All times are stored as exact instants (in UTC). Each browser shows them in its own local time zone.
- **DECIMAL(10,2)** - exact money numbers with 2 decimals (never floating point for storage).
- **JSONB** - a column holding JSON. The prescription's list of medicines is stored like this.
- **Transaction** - a group of changes that succeed together or fail together (all-or-nothing). If anything fails, everything is **rolled back**; otherwise it is **committed**.

2.3.1 Locks - how two people are stopped from booking the same slot

Two patients might press "Book" on the same slot in the same millisecond. Three layers stop that:

- **Row lock** - booking reads the slot with `SELECT ... FOR UPDATE` inside a transaction. The second request has to *wait* until the first commits; then it sees the slot is "booked" and fails with SLOT_UNAVAILABLE.
- **Advisory lock** - `pg_advisory_xact_lock(hashtext('patient:<id>'))` is a named lock Postgres holds until the transaction ends. One per patient serialises that patient's bookings (so two tabs cannot create overlapping appointments). One per doctor (`doctor-slots:<id>`) stops two tabs creating overlapping slots.
- **FOR UPDATE SKIP LOCKED** - used by the reminder job: "lock the rows nobody else has locked, skip the others". So two copies of the server never send the same reminder twice.

2.3.2 The partial unique index (last line of defence)

```
CREATE UNIQUE INDEX appointments_one_active_per_slot ON appointments (slot_id)
WHERE status IN ('pending_confirmation', 'confirmed');
```

Even if the application code had a bug, Postgres itself refuses a second *active* appointment for the same slot. Cancelled/completed ones do not count, so a slot can be re-booked after cancellation. If this ever fires, the server turns the error into the friendly SLOT_UNAVAILABLE message.

2.3.3 The tables (database schema)

```
users --1:1-- doctor_profiles --1:n-- slots
  |                 |                   |
  | 1:n             | 1:n               | 1:n (history)
  +------------ appointments -----------+
                    | 1:1 prescriptions
                    | 1:n reminders
users --1:n-- notifications (optional link to an appointment)
settings  (exactly one row: branding + commission)
```

Table	Important columns	Notes
users	id, name, email (unique, lower-cased), phone, password_hash, role, is_active, created_at, updated_at	Everyone: patients, doctors, admins. Password is stored only as a bcrypt hash.
doctor_profiles	id, user_id (unique FK), specialization, qualification, experience_years, fee, bio, clinic_address, certificate_path/name/mime_type, status, rejection_reason, reviewed_at, reviewed_by	One per doctor user. CHECK fee >= 0. Indexed by status and specialization.

Table	Important columns	Notes
slots	id, doctor_id, start_at, end_at, status	Unique (doctor_id, start_at). CHECK end_at > start_at.
appointments	id, user_id, doctor_id, slot_id (nullable), start_at, end_at, status, reason, doctor_fee, commission, total_fee, confirmed_at, completed_at, cancelled_at, cancelled_by, cancel_reason, reschedule_count, auto_reschedule_count, previous_start_at, doctor_notes	Keeps its own copy of times and prices. Partial unique index on slot_id.
prescriptions	id, appointment_id (unique), diagnosis, medicines (JSONB), advice, follow_up_date	At most one per appointment.
notifications	id, user_id, appointment_id, type, title, message, read_at	In-app messages. read_at null = unread.
reminders	id, appointment_id, type (confirm/day/hour/quarter), send_at, status (pending/sent/skipped), sent_at	Future messages, stored so they survive restarts.
settings	id (=1), app_name, tagline, logo_path, commission_type, commission_value, currency, support_email	Single row, enforced by CHECK (id = 1).
SequelizeMeta	name	Created by sequelize-cli; records which migrations have run.

2.4 Sequelize - talking to Postgres from JavaScript (ORM)

Writing raw SQL everywhere is tedious, so the backend uses **Sequelize**, an **ORM** (Object-Relational Mapper). Each table gets a JavaScript class called a **model** (User, DoctorProfile, Slot, Appointment, Prescription, Notification, Reminder, Setting). Instead of SQL you write `User.findByPk(id)` or `Slot.findAll({ where: { status: 'available' } })` and Sequelize writes the SQL. The `pg` package is the low-level driver that actually sends the SQL to Postgres; `pg-hstore` is a helper Sequelize needs for Postgres.

- **underscored: true** - JavaScript uses camelCase (`startAt`), the database uses snake_case (`start_at`). Sequelize converts automatically.
- **Associations** - `hasOne / hasMany / belongsTo` describe the links, so you can *include* related rows, e.g. load an appointment with its doctor and patient in one query (a SQL JOIN).
- **Scopes** - the User model's default scope hides `passwordHash`; only `User.scope('withPassword')` (used by login and change-password) can read it.
- **toJSON()** - controls what is sent to the browser: User removes passwordHash; DoctorProfile removes the private certificate path and adds `hasCertificate`; Setting turns logoPath into a public `logoUrl`.
- **Op** - operators for queries: Op.in, Op.lt (less than), Op.gte, Op.iLike (case-insensitive LIKE, used for search), Op.or...
- **Migrations** - versioned scripts that create/alter tables. `sequelize-cli db:migrate` runs the ones not yet recorded in SequelizeMeta. This project has one migration that creates everything.

2.5 Authentication: passwords, bcrypt and JWT

2.5.1 Passwords are never stored - bcrypt

When you register, the server runs your password through **bcrypt** (library `bcryptjs`, 10 "rounds"). bcrypt is a deliberately *slow* one-way hash with a random **salt**: the same password gives a different hash each time, and the hash cannot be turned back into the password. Only the hash goes into `users.password_hash`. At login, `bcrypt.compare(typedPassword, storedHash)` answers yes/no. If the database leaked, attackers would still not have the passwords.

Timing protection: if the e-mail does not exist, the server still compares against a dummy hash (`DUMMY_HASH`), so "unknown e-mail" and "wrong password" take the same time and return the same message. Otherwise an attacker could discover which e-mails are registered by measuring response times.

2.5.2 What a JWT is

After login the server must remember who you are on every following request. HTTP itself forgets everything between requests. This project uses a **JSON Web Token (JWT)**: a signed "ID card" the server gives you, and

that you show on every request.

A JWT is three Base64URL-encoded parts joined by dots: `header.payload.signature`.

```
header    {"alg":"HS256","typ":"JWT"}
payload   {"sub":"a3cae773-...","role":"user","iat":1762921000,"exp":1763525800}
signature HMAC-SHA256( base64(header) + "." + base64(payload), JWT_SECRET )

-> eyJhbGciOiJIUzI1NiIs... . eyJzdWIiOiJhM2NhZTc3My... . 3q2-7vG0aKx...
```

- **sub** (subject) = the user's id. **role** = their role. **iat** = issued-at time. **exp** = expiry (now + JWT_EXPIRES_IN, which is 7 days).
- The payload is only *encoded*, not encrypted - anyone can read it. That is fine because it holds no secrets (no password).
- The **signature** is made with `JWT_SECRET`, a long random string in `server/.env` that only the server knows. If anyone changes even one character of the payload (e.g. role "user" -> "admin"), the signature no longer matches and the server rejects the token. Without the secret nobody can make a valid token.
- HS256 = HMAC with SHA-256, a symmetric algorithm: the same secret signs and verifies.

2.5.3 The complete JWT flow in this project

```
1. Browser  POST /api/auth/login {email, password}
2. Server   finds user (withPassword scope) -> bcrypt.compare -> checks isActive
3. Server   signToken(user) = jwt.sign({sub: user.id, role: user.role}, JWT_SECRET, {expiresIn: '7d'})
4. Server   responds {success:true, data:{token, user}}
5. Browser  AuthContext.startSession -> tokenStore.set(token) -> localStorage['docbook.token']
6. Every later request: axios interceptor adds header
               Authorization: Bearer eyJhbGciOi...
7. Server   authenticate middleware:
              - no "Bearer " header            -> 401 UNAUTHORIZED
              - jwt.verify fails (bad/expired) -> 401 TOKEN_INVALID "Your session has expired"
              - loads the user FRESH from the DB by payload.sub (with doctorProfile)
              - user missing                    -> 401 ACCOUNT_NOT_FOUND
              - user.isActive false             -> 401 ACCOUNT_INACTIVE
              - otherwise req.user = user; next()
8. requireRole('admin') etc. then checks req.user.role -> 403 if wrong
9. Browser  on any 401 for a signed-in request: the axios response interceptor calls the
            unauthorized handler -> token removed, user set to null, toast shown, redirect to /login
10. Page reload: AuthProvider sees a saved token -> GET /api/auth/me -> restores the user
11. Logout: the token is deleted from localStorage (JWTs are stateless;
            nothing is stored on the server)
```

Why load the user fresh on every request? The token says "role: user" and stays valid for 7 days. If an admin deactivates the account, or a doctor gets approved, the token would be out of date. Because the middleware reads the user from the database each time, deactivation takes effect on the very next request and the doctor's current approval status is always used. The role inside the token is not trusted for permissions - the role from the database is.

Where the token is kept: in `localStorage` under the key `docbook.token`, so you stay signed in after closing the tab. Trade-off: any JavaScript running on the page could read it, which is why the app avoids risky content (for example SVG logos are refused). If localStorage is blocked, the code catches the error and the session simply lasts until the tab closes.

Secret handling: `config.js` refuses to start in production without JWT_SECRET. In development it falls back to an insecure default and prints a warning. `start.bat` generates a random 96-hex-character secret on first run with `crypto.randomBytes(48)`.

2.6 Express and middleware

Express is the web framework for the backend. A request passes through a chain of **middleware** functions `(req, res, next)`; each can inspect or change the request, reject it, or call `next()` to pass it on. A route like `router.post('/appointments', authenticate, patientOnly, validate({body}), h(appointment.book))` reads left to right: check token -> check role -> validate body -> run the controller.

Middleware / library	What it does here
helmet	Adds security HTTP headers (no sniffing, frame protection...). crossOriginResourcePolicy is "cross-origin" so the logo can be shown from another origin.
cors	Cross-Origin Resource Sharing: only origins in ALLOWED_ORIGINS (default http://localhost:5173) may call the API from a browser. Requests with no Origin (curl, same-origin) are allowed.
express.json / urlencoded	Parse request bodies, max 200 KB.
morgan	Logs each request (method, path, status, time) in development.
express-rate-limit	30 requests per 15 minutes per IP on register/login/change-password, against password guessing. Returns 429 RATE_LIMITED.
multer	Receives uploaded files (multipart/form-data) and saves them to disk.
zod	Validates and cleans request data against schemas.
express.static	Serves files in uploads/branding (logos) publicly, cached 7 days. Certificates are NOT served this way.
trust proxy 1	Trusts one reverse proxy, so the real client IP is used by the rate limiter when deployed behind one.

2.7 Zod - input validation

Never trust what the browser sends. **Zod** describes the allowed shape of the data and `schema.parse(data)` either returns a cleaned copy or throws a ZodError. The cleaning includes: trimming spaces, lower-casing e-mails, converting "800" into the number 800 (*coerce*), turning "" into null, adding defaults, and **stripping any extra fields** (so nobody can sneak in `role: "admin"` on registration). The error middleware turns a ZodError into a 400 VALIDATION_ERROR with a list of field messages.

Password rule (server and client): 8-72 characters, at least one letter and one number. 72 is bcrypt's maximum input length. Phone: optional, 7-20 digits/spaces/dashes with optional +.

2.8 File uploads - multer and safety checks

- Certificates: PDF, JPG or PNG, max 5 MB, form field `certificate`, saved in `server/uploads/certificates`.
- Logos: PNG, JPG or WEBP, max 2 MB, field `logo`, saved in `server/uploads/branding`. **SVG is refused** because SVG files can contain scripts.
- The stored file name is random: `<timestamp>-<16 hex chars>.<ext>`. The extension comes from the accepted type, never from the user's file name, so nobody can upload "evil.html".
- **Magic-number check**: the declared type comes from the browser and can lie, so the server reads the first bytes: PDF starts with `%PDF-`, PNG with `89 50 4E 47 0D 0A 1A 0A`, JPEG with `FF D8 FF`, WEBP with RIFF....WEBP.
- If the request fails after the file was saved, the error handler deletes the file (no orphans).
- Certificates are private: only an admin or the doctor who owns it can download it, through `GET /api/doctors/:id/certificate`, with `Cache-Control: private, no-store`. `path.basename` stops path tricks like "../../".

2.9 Background jobs - node-cron

node-cron runs a function on a schedule. `'* * * * *'` means every minute. The job (`jobs/scheduler.js`) first handles unconfirmed appointments, then sends due reminders. It also runs once 2 seconds after start-up, to catch up on anything missed while the server was off. A `running` flag prevents a slow run overlapping the next one. Turn it off with `SCHEDULER_ENABLED=false`.

2.10 E-mail - nodemailer

Every notification is stored in the database (in-app) and also e-mailed if SMTP settings are filled in (`SMTP_HOST, SMTP_USER, SMTP_PASS...`). Without SMTP, in development the e-mail is only logged as `[mail:dev] to=... subject="..."`. Inside a transaction the e-mail waits for **afterCommit**, so a booking that is rolled back

never sends a "booked" e-mail. E-mail failures are logged and never break the request.

2.11 dotenv and the .env file

Settings and secrets are kept out of the code in `server/.env` (never committed - it is in .gitignore). **dotenv** loads it into `process.env`. `.env.example` is the committed template listing every key:

Key	Default	Purpose
NODE_ENV	development	production turns off request logs and dev e-mail logs, and requires JWT_SECRET
PORT	5000	API port
ALLOWED_ORIGINS	http://localhost:5173	CORS allow-list, comma separated
DB_HOST / DB_PORT	127.0.0.1 / 5432	Where Postgres runs
DB_NAME	doctor_appointments	Database name (the test environment adds _test)
DB_USER / DB_PASSWORD	postgres / postgres	Database login
DB_SSL	false	true for hosted databases that need SSL
DB_LOGGING	false	true prints every SQL query
JWT_SECRET	(empty)	Secret that signs tokens - must be long and random
JWT_EXPIRES_IN	7d	Token lifetime
DISPLAY_TIMEZONE	Asia/Kolkata	Time zone used for times written inside notification texts
SCHEDULER_ENABLED	true	Run the every-minute job
ADMIN_EMAIL / ADMIN_PASSWORD	admin@docbook.local / Admin@1234	First admin created by the seed script
SMTP_HOST/PORT/SECURE/USER/PASS, MAIL_FROM	empty / 587 / false	Optional e-mail sending
LOCK_WINDOW_HOURS	24 (not in the example)	Optional override of the 24-hour rule

2.12 The frontend stack

Tool	What it is
React 18	Library for building the UI out of components (functions that return JSX, HTML-like syntax). State changes re-render the screen.
Hooks	useState (local state), useEffect (run code after render, e.g. load data), useContext (shared data), useMemo/useCallback (avoid needless recomputation), useRef.
Context	Shares data with every component without passing it down by hand: AuthContext (the signed-in user), BrandingContext (app name/logo/currency), CountsContext (sidebar badges).
React Router 6	Maps URLs to pages (/dashboard, /doctors/:id...) without reloading the page. Navigate redirects, useParams reads :id, NavLink highlights the active menu item.
Vite	Development server with instant reload, and the production bundler (npm run build -> dist/). Its proxy forwards /api and /uploads to port 5000.
Tailwind CSS	Utility CSS classes written directly in JSX (p-4, rounded-xl, text-ink). PostCSS + autoprefixer process it. Dark mode is the "dark" class on html.
axios	HTTP client used to call the API, with interceptors that add the token and react to 401.
framer-motion	Animations (toasts, modals, drawers).
lucide-react	The icon set (Stethoscope, Bell, Calendar...).
Google Fonts	Plus Jakarta Sans and JetBrains Mono, loaded in index.html.

Part 3 - End-to-end flows (what happens, step by step)

3.1 Starting the app (start.bat)

- Sets PG_HOME (default %USERPROFILE%\pgsql). If `pg_ctl.exe` exists there, checks Postgres with `pg_isready`; if it is not running, starts it with `pg_ctl -D data -l postgres.log start -w`. Otherwise it assumes you run your own Postgres on 127.0.0.1:5432.
- If server/node_modules or client/node_modules are missing, runs `npm install` there.
- If server/.env is missing: copies .env.example, generates a random JWT_SECRET and writes it in with PowerShell, then runs `npm run db:setup` (create DB, migrate, seed demo data). Stops with a message if that fails.
- Opens two new console windows: "Backend - API" (`npm run dev` in server, using nodemon) and "Frontend - React" (`npm run dev` in client, Vite).
- Waits 6 seconds, opens http://localhost:5173 in the browser and prints the demo logins.

On server start (`server.js`): connect to Postgres (`sequelize.authenticate()`); if that fails, print "Could not connect to PostgreSQL" and exit. Otherwise listen on PORT, start the scheduler, and on Ctrl+C (SIGINT/SIGTERM) close the HTTP server and DB pool cleanly (forced exit after 10 s).

3.2 Opening the website

- `index.html` loads `main.jsx`, which renders `<App/>` inside React StrictMode.
- App wraps everything in BrowserRouter -> BrandingProvider -> AuthProvider, and mounts ConfirmHost (confirmation dialogs) and ToastHost (pop-up messages).
- BrandingProvider calls `GET /api/settings/public` (app name, tagline, logo, currency, commission) and sets the browser tab title. If it fails, defaults are used ("DocBook", INR).
- AuthProvider: if a token is saved, calls `GET /api/auth/me` to restore the user (a spinner shows meanwhile); a failure removes the token.
- The Landing page (/) shows the brand, features and 6 doctors from `GET /api/doctors?limit=6`. Signed-in users are redirected to /dashboard.

3.3 Patient registration and login

- Register page checks fields in the browser (name, valid e-mail, password rule, phone) and sends `POST /api/auth/register`.
- Server: rate limit -> zod validation -> bcrypt hash -> `User.create(role user)` -> response `{token, user}`. Duplicate e-mail -> 409 "An account with this email already exists".
- Client stores the token, sets the user, shows "Welcome!" and goes to /doctors.
- Login: `POST /api/auth/login`. On success, returns to the page you came from (saved by RequireAuth in location.state.from) or /dashboard. The login page has one-click demo buttons.

3.4 Doctor registration and approval

- RegisterDoctor collects account fields, practice fields (specialization, qualification, experience, fee, address, bio) and a certificate file, and sends one `multipart/form-data` request to `POST /api/auth/register/doctor`.
- Server: rate limit -> multer saves the file (type, size, magic bytes checked) -> zod validates the text fields (numbers coerced from strings) -> in one transaction: create User (a leading "Dr." is removed from the name), create DoctorProfile (status pending), notify every active admin "New doctor awaiting approval".
- The doctor is signed in immediately but sees only Dashboard, Notifications and My profile. The dashboard explains the profile is under review. Slot routes return 403 DOCTOR_NOT_APPROVED.
- Admin opens Doctors (pending ones are listed first, oldest first), opens the review drawer, previews the certificate inline (fetched as a blob with the token and shown in an iframe/img), then Approves, or Rejects with a required reason.

- Server `reviewDoctor`: cannot approve without a certificate, cannot set the same status twice, saves reviewedAt/reviewedBy, notifies the doctor. Revoking an approved doctor cancels all their future appointments and notifies the patients.
- A rejected doctor uploads a new certificate (`PUT /api/doctor/profile/certificate`): the old file is deleted, status goes back to pending, rejection reason cleared.

3.5 Doctor publishes availability

- Availability page -> "Add availability": date range, weekdays, start/end hour, slot length, gap between slots. `buildSlots()` turns this into exact start/end instants in the doctor's **browser time zone** and converts them to UTC ISO strings (skipping past times, max ~400 previewed).
- `POST /api/doctor/slots {slots:[...]}` (1-300 per request). The server skips slots that are in the past, shorter than 10 or longer than 240 minutes, more than 90 days ahead, overlapping each other in the request, or overlapping existing slots (checked under a per-doctor advisory lock). It creates the rest and returns `{created, skipped}`.
- The week view shows each day's slots: free ones can be deleted; booked ones show the patient and open the appointment; blocked ones can be reopened. A booked slot cannot be deleted (409 SLOT_BOOKED).

3.6 Patient searches and books

- Find doctors: a search box (debounced 300 ms) and a specialization filter (`GET /api/doctors/specializations` gives names with counts). `GET /api/doctors` returns approved, active doctors only, sorted by soonest free slot (doctors with nothing free last), each with `pricing` = {doctorFee, commission, totalFee}.
- Doctor page: profile, fee breakdown and free slots for the next 30 days (`GET /api/doctors/:id/slots`), grouped by local day.
- Selecting a slot shows a notice: within 24 h -> "confirmed immediately and cannot be cancelled or rescheduled"; otherwise -> "confirm before <start - 24 h>".
- `POST /api/appointments {slotId, reason}`. Server, in one transaction: patient advisory lock -> lock slot row FOR UPDATE -> must be available and in the future -> doctor must be approved and active -> patient must not have an overlapping active appointment -> calculate fees from current settings -> create appointment (confirmed if inside 24 h, else pending_confirmation) -> mark slot booked -> create reminders -> notify patient and doctor.
- The client is taken to the appointment page.

3.7 Confirm, "I can't make it", cancel

- **Confirm** (`POST /:id/confirm`): only the patient, only while pending and before the deadline. Idempotent (confirming twice is fine). Removes the "confirm" reminder and notifies the doctor. After the deadline -> 422 CONFIRMATION_CLOSED.
- **I can't make it**: the modal calls `GET /:id/reschedule-preview`, which finds the doctor's earliest free slot after the current one where the patient has no clash. The patient sees "Current -> New" and presses Move -> `POST /:id/reschedule {slotId}`. The old slot is freed and the new one booked in the same transaction; the new time counts as confirmed; reminders are rebuilt; both sides notified. If someone took the slot meanwhile (SLOT_UNAVAILABLE), the modal fetches the next one automatically. No later slot -> option to cancel instead.
- **Cancel**: the patient can cancel only outside 24 h (reason optional) and the slot reopens. A doctor or admin can cancel any active appointment that has not started, but must give a reason. When the **doctor** cancels, the slot becomes *blocked* (they are unavailable then).
- After every action the server returns the fresh appointment, including `actions`, and the page just re-renders. If an action fails with LOCK_WINDOW / INVALID_STATUS / CONFIRMATION_CLOSED, the client re-fetches the appointment because it may have changed underneath (e.g. moved by the scheduler).

3.8 The every-minute job

Step 1 - processUnconfirmed: finds up to 50 appointments still pending_confirmation whose start is within 24 h. For each, in its own transaction (patient lock, row lock with SKIP LOCKED, re-check): if it has been moved automatically 2 times already -> cancel. Otherwise find the doctor's next free slot starting at least 48 h from now with no patient clash -> move it there (status stays pending, autoRescheduleCount+1, reminders reset) and notify both. No slot -> cancel ("Not confirmed in time and no later slot was available"), which frees the slot. If the chosen slot was taken a moment ago, it simply tries again next minute.

Step 2 - processDue (reminders): claims up to 100 due pending reminders with FOR UPDATE SKIP LOCKED. A reminder is *skipped* if the appointment is gone, not active, already started, a confirm-nudge for an already confirmed booking, or a pending booking past its deadline (it is about to move). Otherwise it notifies the patient (and for the 1-hour reminder also the doctor) and marks it sent.

Reminder	Sent at	To
confirm	start - 36 h (24 h lock + 12 h nudge), only for pending bookings	Patient
day	start - 24 h	Patient
hour	start - 1 h	Patient and doctor
quarter	start - 15 min	Patient

Reminders are created only if their time is still in the future (a booking 5 h ahead gets only hour + quarter).

3.9 The visit, completion and prescription

- Complete / No-show buttons appear only for the doctor, only for a confirmed appointment, and only once it has **started**. Complete accepts private notes (never sent to the patient - removed in `present()`).
- Prescription (`PUT /:id/prescription`): diagnosis, up to 30 medicines (name, dosage, frequency, duration, instructions), advice, follow-up date (YYYY-MM-DD). At least one of diagnosis/medicine/advice is required. Saving again updates it. The patient is notified and can view and print it (print styles hide the rest of the page).
- Earnings: the doctor dashboard sums doctorFee of completed appointments; the admin dashboard sums commission and totalFee of completed appointments.

3.10 Notifications and badges

The AppShell polls `GET /api/notifications/unread-count` every 30 seconds (only while the tab is visible); for admins it also counts pending doctors. The counts show as red badges in the sidebar. The Notifications page lists all/unread messages (paginated); clicking one marks it read and opens its appointment (or the doctor queue for "doctor_registered"). "Mark all read" is available.

3.11 Admin: users and settings

- Users: filter by role and active/inactive, search name/e-mail/phone. Deactivating asks for confirmation, cannot target yourself, and cancels the user's future appointments (patient) or all of a doctor's future appointments, notifying the other side. Deactivated users are rejected on their very next request.
- Settings - Branding: app name, tagline, support e-mail, logo upload/remove (old logo file deleted). Commission: percentage (max 100) or flat amount, and currency (INR, USD, EUR, GBP, AED, SGD, AUD, CAD), with a live fee preview. Only new bookings use new values.

Part 4 - Every file: project root and server

4.1 Project root

`README.md`

Project overview: tech stack, quick start, manual setup, demo accounts, test instructions, features by role, business rules, edge cases, database diagram, API overview and folder structure. Small inaccuracy: it mentions "stop-db.bat", but the file is actually named stop.bat; the last three lines repeat the demo logins.

`start.bat`

Windows one-click launcher (explained step by step in 3.1). Uses `setlocal` so variables do not leak, `cd /d "%~dp0"` to run from its own folder, `call npm ...` so the batch file continues after npm, and `start "title" cmd /k` to open windows that stay open.

`stop.bat`

Stops the portable PostgreSQL with `pg_ctl -D data stop -m fast` ("fast" = disconnect clients, then stop cleanly). The Backend and Frontend windows are closed by hand.

`.gitignore`

Files git must not track: node_modules/, .env, .env.local, dist/, uploaded files (server/uploads/* except .gitkeep) and *.log.

4.2 server/ (configuration files)

`server/package.json`

Backend name, scripts and libraries. Scripts: `start` (node), `dev` (nodemon - restarts on changes in src, ignores uploads), `db:create`, `db:migrate`, `db:migrate:undo` (drop everything), `db:seed` (admin + settings), `db:seed:demo` (+ demo data), `db:setup` (create+migrate+demo seed), `db:reset` (undo+migrate+seed), `test:e2e`.

`server/package-lock.json`

Exact installed versions of every package (generated by npm; never edited by hand).

`server/.env.example / server/.env`

Template and real settings file (see 2.11). .env holds your real DB password and JWT secret and is never committed.

`server/.sequelizerc`

Tells sequelize-cli where things are: config = src/config/database.js, models, migrations and seeders folders.

`server/uploads/`

Created at runtime: certificates/ (private) and branding/ (public logos). Only .gitkeep is committed.

4.3 server/src - entry points

`src/server.js`

Starts the backend: connects to the DB, starts listening, starts the scheduler, handles shutdown signals (3.1).

`src/app.js`

Builds the Express app: disables the X-Powered-By header, trusts one proxy, helmet, CORS allow-list, JSON/urlencoded parsers (200 KB), morgan in dev, public static route `/uploads/branding` (fallthrough:false so a missing logo gives 404), mounts all routes under `/api`, then the 404 handler and the error handler. Exported separately from server.js so it can be loaded without listening.

4.4 server/src/config

`config/config.js`

Loads .env and builds one config object: NODE_ENV, IS_PRODUCTION, PORT, ALLOWED_ORIGINS (split by commas), JWT_SECRET/EXPIRES, UPLOAD_DIR, DISPLAY_TIMEZONE, SCHEDULER_ENABLED, SMTP, ADMIN_EMAIL/PASSWORD, and RULES: LOCK_WINDOW_HOURS 24, CONFIRM_NUDGE_HOURS 12, AUTO_RESCHEDULE_MIN_LEAD_HOURS 48, MAX_AUTO_RESCHEDULES 2, MIN/MAX_SLOT_MINUTES 10/240, MAX_SLOTS_PER_REQUEST 300, MAX_SLOT_RANGE_DAYS 90. Throws in production if JWT_SECRET is missing; warns in development. The helper `num()` turns env text into numbers with a fallback.

`config/constants.js`

Frozen (unchangeable) lists of names used everywhere: ROLES, DOCTOR_STATUS, SLOT_STATUS, APPOINTMENT_STATUS, ACTIVE_APPOINTMENT_STATUSES (pending_confirmation + confirmed = "holds a slot"), REMINDER_TYPES, COMMISSION_TYPES. Keeping them in one place prevents typos.

`config/database.js`

Database connection settings shared by the app and sequelize-cli: user, password, database, host, port, dialect postgres, optional SQL logging, underscored names, connection pool (max 10 connections, idle 10 s), optional SSL. Exports development, test (database name + "_test") and production.

4.5 server/src/models

`models/index.js`

Creates the single Sequelize connection from database.js, loads the 8 models, calls each model's `associate()` to wire up relationships, and exports `{sequelize, Sequelize, User, DoctorProfile, ...}`.

`models/User.model.js`

Users table. E-mail setter trims and lower-cases. Default scope hides passwordHash; "withPassword" scope includes it; toJSON removes it. Relations: hasOne doctorProfile, hasMany appointments and notifications.

`models/DoctorProfile.model.js`

Doctor details, certificate info, status, review info. `fee` getter returns a JS number (Postgres DECIMAL arrives as a string). toJSON hides certificatePath and adds hasCertificate. Relations: belongsTo user and reviewer, hasMany slots and appointments.

`models/Slot.model.js`

A doctor's bookable time range with status. belongsTo doctor, hasMany appointments (history: a slot can have old cancelled appointments).

`models/Appointment.model.js`

The booking: patient, doctor, slot (nullable - if a slot row is deleted the appointment keeps its own times), status, reason, price snapshot (doctorFee/commission/totalFee as numbers), timestamps for confirm/complete/cancel, who cancelled and why, reschedule counters, previousStartAt, private doctorNotes. Relations to patient, doctor, slot, prescription, reminders.

`models/Prescription.model.js`

One per appointment: diagnosis, medicines JSONB array, advice, followUpDate (date only).

`models/Notification.model.js`

In-app message for a user, optionally linked to an appointment. readAt null = unread.

`models/Reminder.model.js`

A message due at sendAt (type confirm/day/hour/quarter), status pending/sent/skipped. Stored rather than computed so it survives restarts and is sent once.

`models/Setting.model.js`

The single settings row (id 1): appName (default DocBook), tagline, logoPath, commissionType (default percentage), commissionValue (default 10), currency (default INR), supportEmail. toJSON exposes logoUrl instead of logoPath.

4.6 server/src/migrations

migrations/20261111000001-create-schema.js

Creates all 8 tables in **one transaction** (if anything fails, nothing is created), with UUID keys defaulting to gen_random_uuid(), created_at/updated_at defaulting to NOW(), foreign keys with CASCADE/SET NULL, indexes (users.role; doctor_profiles.status and specialization; unique slots(doctor_id,start_at); slots(status,start_at); appointments by user/doctor/status + start; notifications by user+created and user+read; reminders by status+send_at and appointment), CHECK constraints (fee >= 0, end > start, settings id = 1, commission >= 0) and the partial unique index against double booking. `down()` drops all tables and the ENUM types.

4.7 server/src/middlewares

middlewares/auth.middleware.js

`signToken` (creates the JWT), `authenticate` (verifies it and loads the user fresh - see 2.5.3), `requireRole(...roles)` (403 if the role does not match) and `requireApprovedDoctor` (403 DOCTOR_NOT_APPROVED unless the doctor's profile is approved).

middlewares/validate.middleware.js

`validate({params, query, body})` parses each part with its zod schema and **replaces** it with the cleaned version; errors go to the error handler.

middlewares/upload.middleware.js

Builds the multer uploaders (certificateUpload, logoUpload) with random names, size limits, MIME allow-lists, friendly errors (FILE_TOO_LARGE, UNEXPECTED_FILE, INVALID_FILE_TYPE) and the magic-number check (INVALID_FILE_CONTENT). Exports `removeFile` (delete, ignoring errors).

middlewares/error.middleware.js

`notFoundHandler`: 404 ROUTE_NOT_FOUND for unknown paths. `errorHandler`: deletes an uploaded file if the request failed, then maps errors: ZodError -> 400 VALIDATION_ERROR with field details; AppError -> its own status/code; unique violation -> 409 DUPLICATE ("account with this email already exists"); foreign-key violation -> 409 IN_USE; MulterError -> 400 UPLOAD_ERROR; broken JSON -> 400 INVALID_JSON; anything else -> logged and 500 INTERNAL with a generic message.

4.8 server/src/validators

validators/index.js

All zod schemas. Shared pieces: trimmed text, optional text ("" -> null), e-mail, password, phone, name (2-120), uuid, ISO date, money (0 - 1,000,000). Schemas: auth (registerUser, registerDoctor, login, updateMe, changePassword); doctor (publicList, range from/to, updateProfile - partial but not empty, createSlots 1-300); appointment (list filters status/scope/search/doctorId/from/to/page/limit, book, reschedule, cancel reason max 300, complete notes max 2000, prescription with medicines max 30 and the "at least something" rule); notification list; admin (doctors filter, reviewDoctor approved|rejected + reason, users filter, userStatus boolean, settings - percentage max 100, 3-letter currency).

4.9 server/src/utils

utils/AppError.js

An Error with an HTTP status, a stable code and optional details, plus shortcuts: badRequest 400, unauthorized 401, forbidden 403, notFound 404, conflict 409, rule 422.

utils/helpers.js

MINUTE/HOUR/DAY in milliseconds; `asyncHandler` (forwards errors from async handlers to Express - Express 4 does not do this by itself); `formatDateTime` (en-IN, DISPLAY_TIMEZONE); `roundMoney` (2 decimals); `parsePagination` (page >= 1, limit default 20, max 100) and `pageMeta` (page, limit, total, totalPages); `ok()` (sends {success:true, data, ...extra}).

utils/fees.js

`calculateFee(doctorFee, settings)`: percentage -> commission = fee x value / 100; flat -> commission = value; total = fee + commission; all rounded. Example: fee 800, 10% -> 800 + 80 = 880. Server-only, and copied onto the appointment.

4.10 server/src/services (business logic)

services/settings.service.js

`getSettings()` returns the single settings row, creating it with defaults the first time (findOrCreate id 1).

services/auth.service.js

registerUser, registerDoctor (transaction + notify admins), login (timing-safe, rejects inactive with 403 ACCOUNT_INACTIVE), loadSession (fresh user + token), updateMe (name/phone), changePassword (checks the current password first).

services/doctor.service.js

Public: listPublic (approved + active only; search across name, specialization, qualification; sorted by nextAvailableAt, a sub-query for the earliest free future slot; adds pricing; hides rejectionReason/reviewedBy and the certificate path), specializations (with counts), getPublic, availableSlots (max 60-day range, never in the past). Doctor: updateProfile (user + profile in one transaction), replaceCertificate (only when not approved), listOwnSlots (with appointment + patient), createSlots (see 3.5), deleteSlot (locked; not if booked), unblockSlot (only blocked and future).

services/appointment.service.js

The heart of the system. Helpers: isInsideLockWindow, confirmDeadline, lockPatient (advisory lock), loadLocked (row lock; relations loaded separately because Postgres cannot combine FOR UPDATE with outer joins), assertParticipant (404 if not yours), assertPatientCanChange, findUserClash (overlap = A.start < B.end AND A.end > B.start), findNextSlot (checks up to 25 candidates). `present()` adds confirmBy, isLocked and the **actions** object (confirm, reschedule, cancel, complete, noShow, prescribe) for the current viewer and strips doctorNotes for patients - so the UI never re-implements the rules. Operations: book, confirmAttendance, previewReschedule, reschedule, moveToSlot, cancel, cancelInTransaction, complete, markNoShow, savePrescription, list (role-scoped; scope upcoming/past/all; status; search by patient or doctor name; date range; admin doctor filter), getById, processUnconfirmed, cancelFutureFor.

services/reminder.service.js

SCHEDULE of reminder offsets and MESSAGES texts; scheduleFor (create future reminders), cancelPending (delete unsent ones, optionally by type), resetFor (after a time change), processDue (send/skip, see 3.8).

services/notification.service.js

`notify(userId, {type,title,message,appointmentId}, transaction)`: stores the notification and e-mails it (after commit if inside a transaction; only to active users).

services/mailer.service.js

Creates a nodemailer transport only if SMTP host, user and password are set; `sendMail` sends, or logs in development.

services/admin.service.js

listDoctors (pending first), getDoctor (+ appointment counts by status), reviewDoctor, certificateFile, listUsers, setUserActive (with cascades), updateSettings (percentage <= 100), updateLogo (replace and delete the old file), removeLogo.

services/dashboard.service.js

forActor picks by role. Patient: counts by status, next 5 upcoming, number awaiting confirmation, prescriptions, unread notifications, total spent (completed). Doctor (approved): counts, today's list, upcoming count, open future slots, earnings, unique patients; if not approved returns just the status/reason. Admin: patients, doctors by status, appointment counts, today's count, commission and gross revenue (completed), 6 latest bookings, 5 oldest

pending doctors.

4.11 server/src/controllers (thin HTTP handlers)

Controllers only read the request, call a service and send the result with `ok()`. They contain no business rules.

`controllers/auth.controller.js`

registerUser (201), registerDoctor (201), login, me (returns req.user), updateMe, changePassword.

`controllers/doctor.controller.js`

Public list/specializations/get/availableSlots; doctor myProfile, updateProfile, replaceCertificate, mySlots, createSlots (201 if any created, else 200), deleteSlot, unblockSlot.

`controllers/appointment.controller.js`

Each action calls the service and answers with the **fresh** appointment via getById (including actions). book answers 201.

`controllers/notification.controller.js`

list (with unread count in meta), unreadCount, markRead (404 if not yours), markAllRead.

`controllers/admin.controller.js`

Doctors, users, settings and logo handlers, plus downloadCertificate (admin or owning doctor only; streams the file with the right content type and no-store caching).

4.12 server/src/routes and jobs

`routes/index.js`

The full route table (see Part 6), the auth rate limiter, the public /health and /settings/public endpoints, the dashboard endpoint, and the middleware chains per group (doctorOnly, patientOnly, approvedDoctor, adminOnly).

`jobs/scheduler.js`

node-cron every minute: processUnconfirmed then processDue, with an overlap guard and a start-up catch-up run (2.9).

4.13 server/src/scripts

`scripts/seed.js`

Always: ensures the settings row and the admin account. With --demo: patient Priya Sharma; doctors Ananya Rao (Cardiology, 800), Rahul Menon (Dermatology, 600), Fatima Sheikh (Pediatrics, 500) - approved - and Vikram Iyer (Orthopedics, 700) - pending; a small generated demo PDF certificate; 30-minute slots 10:00-13:00 and 16:00-18:00 for the next 10 days except Sundays; and one completed appointment 3 days ago with Dr. Rao, with a prescription. Safe to re-run (findOrCreate; existing doctors are skipped).

`scripts/e2e.test.js`

End-to-end test against the running API (17 steps): doctor registers with upload; wrong file type rejected; unapproved doctor cannot add slots; admin approves; slot creation skips overlaps/past; fee = doctor fee + commission; two patients register; concurrent booking of one slot - exactly one wins; >24 h booking waits for confirmation and price is snapshotted; <24 h booking confirmed and locked; patients cannot see each other's appointments; reschedule preview and move; cancel reopens slot; unconfirmed appointment auto-moved (time is simulated by shifting rows); complete only after start, then prescription; notifications received; a patient cannot act as doctor or admin. Cleans up everything it created.

Part 5 - Every file: client (frontend)

5.1 client/ (configuration files)

`client/package.json`

Frontend libraries and scripts: `dev` (Vite dev server), `build` (production bundle into dist/), `preview` (serve the built bundle). Dependencies: react, react-dom, react-router-dom, axios, framer-motion, lucide-react. Dev: vite, @vitejs/plugin-react, tailwindcss, postcss, autoprefixer.

`client/package-lock.json`

Exact installed versions (generated).

`client/vite.config.js`

Enables the React plugin, port 5173, and the **proxy**: requests to /api and /uploads are forwarded to http://localhost:5000 (or VITE_PROXY_TARGET). Because the browser thinks everything comes from 5173, no CORS is needed in development. This proxy is what printed "http proxy error ECONNREFUSED" when the backend was not running.

`client/.env.example`

`VITE_API_URL` - empty in development; for a separate deployment set it to the API origin (e.g. https://api.example.com).

`client/index.html`

The single HTML page: page title, favicon, Google Fonts, an empty `<div id="root">` and the script tag loading src/main.jsx. React draws everything inside #root (a "single-page application").

`client/tailwind.config.js`

Dark mode by class; which files to scan for class names; fonts; and theme-aware colour names (canvas, surface, subtle, line, ink, ink-2, ink-3, accent) that point to CSS variables, so one class works in both light and dark themes.

`client/postcss.config.js`

Runs Tailwind and autoprefixer (adds browser prefixes) on the CSS.

`client/public/favicon.svg`

The browser-tab icon (served as-is).

5.2 client/src - entry and routing

`src/main.jsx`

Creates the React root on #root and renders <App/> in StrictMode (which double-checks components in development); imports index.css.

`src/index.css`

Tailwind layers plus the CSS variables for the light and dark themes (backgrounds, surfaces, borders, text colours, accent indigo #4f46e5, shadows), base body styles, and reusable classes such as form inputs, the small uppercase "label-eyebrow", the "ledger-table" table style, skeleton loading blocks, and print rules.

`src/App.jsx`

All routes and guards. **RequireAuth**: shows a spinner while restoring the session, otherwise sends guests to /login (remembering where they wanted to go). **RequireRole**: wrong role -> /dashboard; with approvedDoctor, unapproved doctors -> /dashboard. **GuestOnly**: signed-in users cannot see login/register. NotFound page for unknown paths.

Path	Page	Who
/	Landing	Everyone (signed-in -> /dashboard)

Path	Page	Who
/login, /register, /register/doctor	Login, Register, RegisterDoctor	Guests only
/dashboard	DashboardPage	Any signed-in user
/doctors, /doctors/:id	FindDoctors, DoctorDetail	Patients
/appointments, /appointments/:id	AppointmentsPage, AppointmentDetail	Patients, approved doctors, admins
/notifications	NotificationsPage	Any signed-in user
/account	AccountPage	Patients and doctors (incl. pending)
/availability	Availability	Approved doctors
/admin/doctors, /admin/users, /admin/settings	Admin pages	Admins

These client-side guards are only for convenience. The real protection is on the server: even if someone bypassed the React routes, the API would still refuse.

5.3 client/src/context

`context/AuthContext.jsx`

Holds the signed-in user. Provides user, loading, setUser, refresh (GET /auth/me), logout (delete token), login, registerUser, registerDoctor (each stores the token via startSession). Registers the 401 handler that signs the user out with a toast.

`context/BrandingContext.jsx`

Loads public settings, keeps defaults if that fails, sets document.title to the app name, and provides logoSrc (full URL via assetUrl) and refresh (called after the admin changes settings).

5.4 client/src/lib (shared helpers)

`lib/api.js`

The axios instance (baseURL `VITE_API_URL + /api`, 20 s timeout). tokenStore (localStorage key docbook.token, safe if storage is blocked). Request interceptor adds `Authorization: Bearer`. Response interceptor calls the unauthorized handler on 401 for requests that carried a token. Helpers: errorMessage (server message, timeout, network error, fallback), errorCode, assetUrl, unwrap (response -> data).

`lib/useAsync.js`

Hook that loads data and returns {data, meta, loading, error, reload, setData}. It numbers each request and drops answers from older requests, so typing fast in a search box never shows stale results.

`lib/format.js`

money (Intl currency format), dateTime/dateOnly/timeOnly/dayLabel/weekdayShort (en-IN, the viewer's local time zone), relative ("in 3 h", "2 d ago"), dayKey (local YYYY-MM-DD, not UTC), initials, doctorName ("Dr. ..."), and label/colour maps for appointment statuses, doctor statuses and roles.

`lib/validation.js`

Browser-side copies of the server rules: isEmail, passwordProblem, phoneProblem, allowed certificate/logo types, fileProblem (type and size). The server re-checks everything.

`lib/files.js`

certificateUrl fetches a private certificate with the token as a blob and makes a temporary object URL; openCertificate opens a blank tab first (so pop-up blockers allow it), then points it at the file and frees the URL after a minute.

`lib/theme.js`

useDarkMode: starts from the saved choice (localStorage docbook.theme) or the operating system preference, toggles the "dark" class on <html> and saves the choice.

lib/toast.jsx

Tiny global toast system: `toast.success(msg)` / `toast.error(msg)` anywhere; ToastHost shows one animated message (top-right on desktop, full width on phones) for 3.5 s (errors 6 s), dismissible.

lib/ui.jsx

The app's own UI kit (about 600 lines): Panel, PageHeader, SectionHeader, Toolbar, Button (variants primary/success/danger/ghost, sizes, loading spinner, icon), IconButton, Field (label + hint + error), Input, Textarea, Select, SearchInput, SegmentedControl (tab-like filter), Badge and statusTone, StatTile (dashboard numbers), EmptyState, Spinner, SkeletonRows, DataTable (with loading/empty rows and row click), Modal, Drawer (side panel), askConfirm + ConfirmHost (promise-based "Are you sure?" dialog: `if (await askConfirm(...))`), EmptyRow, LoadingRows, RowAction, Switch.

5.5 client/src/components

components/AppShell.jsx

Layout for every signed-in page: the Sidebar, a mobile top bar with a menu button, and the page content (<Outlet/>). Polls the unread count (and pending doctors for admins) every 30 s while visible, shares it through CountsContext (useCounts), scrolls to the top on page change, and asks for confirmation on sign-out.

components/Sidebar.jsx

`menuFor(user, counts)` builds the menu per role (an unapproved doctor sees only Dashboard, Notifications, My profile). Collapsible on desktop, slide-in on mobile, active item highlighted, red count badges, user name/e-mail, theme toggle and sign-out.

components/Brand.jsx

Shows the admin-configured logo (or a stethoscope icon) and app name, optionally a subtitle (the role).

components/AppointmentActions.jsx

The action buttons for one appointment, shown **only if the server's actions flags allow them**: Confirm, I can't make it, Complete, No-show, Cancel; or "Changes closed (within 24 h)". Contains RescheduleModal (preview next slot, move, auto-retry on SLOT_UNAVAILABLE, cancel-instead), CancelModal (reason required for doctor/admin) and CompleteModal (private notes).

components/Prescription.jsx

PrescriptionView (diagnosis, medicines table, advice, follow-up, Print button) and PrescriptionForm (editable medicine rows, add/remove, up to 30, follow-up date not in the past, save/update).

components/FilePicker.jsx

Click-or-drag-and-drop single-file box showing the chosen file name and size, with a remove button and an error line. Keyboard accessible (Enter/Space).

components/common.jsx

AppointmentStatusBadge, DoctorStatusBadge, FeeBreakdown (consultation fee + platform fee = total payable, in the configured currency), Pagination (Previous/Next with "Page x of y"), ErrorBanner (with Retry), Notice (info/warning/success/danger strip), Avatar (initials).

5.6 client/src/pages

pages/Landing.jsx

Public home: header with Sign in / Get started, tagline, four feature cards (verified doctors, real availability, smart reminders, flexible changes) and a preview of 6 doctors with prices.

pages/auth/AuthLayout.jsx

Shared frame for the login/register pages (brand, title, subtitle, card; wide variant for the doctor form).

pages/auth/Login.jsx

E-mail + password form, error message, one-click demo account buttons (Patient, Doctor, Admin), links to both registrations; returns to the page you came from.

pages/auth/Register.jsx

Patient sign-up with live validation; on success goes to Find doctors.

pages/auth/RegisterDoctor.jsx

Doctor sign-up in three sections (Account, Practice, Registration certificate). Builds a FormData with all fields and the file; on success goes to the dashboard (pending review).

pages/DashboardPage.jsx

Loads GET /dashboard and shows one of three dashboards. Patient: banner if bookings await confirmation, upcoming list with inline actions, totals. Doctor: if not approved, the review/rejection status; else today's schedule, a nudge to add slots when none are open, stats and earnings. Admin: stat tiles (patients, doctors + pending, today), appointments by status, doctors awaiting review (click -> review drawer), latest bookings table.

pages/patient/FindDoctors.jsx

Search (debounced), specialization filter, paginated doctor cards with total price and next free time.

pages/patient/DoctorDetail.jsx

Doctor profile, slots for 30 days grouped by day in local time, booking panel with FeeBreakdown, optional reason, the 24-hour notice, and the Book button.

pages/AppointmentsPage.jsx

Role-aware appointment list: scope tabs (upcoming/past/all), status filter, search, paginated table with status badges and inline actions; row click opens the detail page.

pages/AppointmentDetail.jsx

One appointment: StatusNotice (confirm-by deadline, locked notice, cancelled by whom and why, completed), doctor and patient cards (contact details), actions, prescription (form for the doctor when allowed, view for others), payment snapshot ("price fixed at booking time") and a history timeline (booked, confirmed, rescheduled, cancelled, completed).

pages/NotificationsPage.jsx

All/unread filter, paginated list, click to mark read and open, mark-all-read; refreshes the sidebar badge.

pages/AccountPage.jsx

Personal details (name, phone), password change (current + new + confirm). For doctors also: Certificate (status, rejection reason, view the current file, upload a new one if not approved) and Professional profile (specialization, qualification, experience, fee, address, bio - "new fees apply to new bookings only").

pages/doctor/Availability.jsx

Slot generator (buildSlots, with a preview count) and a week view with previous/next week navigation; slot chips for free/booked/blocked with delete, reopen and open-appointment actions.

pages/admin/Doctors.jsx

Doctor queue with status filter tabs and search; ReviewDrawer with details, appointment stats, inline CertificatePreview (PDF in an iframe or image), approve / reject (reason) / revoke (confirmation, cancels future appointments). Supports opening a review directly from ?review=<id>.

pages/admin/Users.jsx

User table with role and status filters, search, and an active/inactive switch (confirmation; reports how many appointments were cancelled).

pages/admin/Settings.jsx

Branding (name, tagline, support e-mail, logo upload/remove) and Commission & currency (percentage/flat, value, currency list, live FeeBreakdown preview, note that existing bookings keep their price).

Part 6 - Reference: API, rules, scripts, security

6.1 Every API endpoint (all under /api)

Method + path	Access	Purpose
GET /health	Public	Server alive check
GET /settings/public	Public	Branding, currency, commission
POST /auth/register	Public, rate-limited	Patient sign-up -> {token, user}
POST /auth/register/doctor	Public, rate-limited, multipart	Doctor sign-up with certificate
POST /auth/login	Public, rate-limited	Login -> {token, user}
GET /auth/me	Signed in	Current user (with doctorProfile)
PATCH /auth/me	Signed in	Update name/phone
POST /auth/change-password	Signed in, rate-limited	Change password
GET /doctors	Public	Approved doctors (search, specialization, page)
GET /doctors/specializations	Public	Specializations with counts
GET /doctors/:id	Public	One approved doctor with pricing
GET /doctors/:id/slots	Public	Free future slots (from/to)
GET /doctors/:id/certificate	Admin or owning doctor	Download the private certificate
GET /doctor/profile	Doctor	Own profile
PATCH /doctor/profile	Doctor	Update profile (+ name/phone)
PUT /doctor/profile/certificate	Doctor (not approved)	Upload a new certificate -> pending
GET /doctor/slots	Approved doctor	Own slots in a range, with bookings
POST /doctor/slots	Approved doctor	Create slots in bulk -> {created, skipped}
DELETE /doctor/slots/:id	Approved doctor	Delete a free/blocked slot
POST /doctor/slots/:id/unblock	Approved doctor	Reopen a blocked future slot
GET /appointments	Signed in (scoped by role)	List with filters
POST /appointments	Patient	Book a slot
GET /appointments/:id	Participant or admin	Detail with actions
POST /appointments/:id/confirm	Patient	Confirm attendance
GET /appointments/:id/reschedule-preview	Patient	Next free slot
POST /appointments/:id/reschedule	Patient	Move to a slot
POST /appointments/:id/cancel	Participant or admin	Cancel (reason required for doctor/admin)
POST /appointments/:id/complete	Approved doctor (own)	Mark completed (+ notes)
POST /appointments/:id/no-show	Approved doctor (own)	Mark no-show
PUT /appointments/:id/prescription	Approved doctor (own)	Create/update prescription
GET /notifications	Signed in	List (unread filter, page)
GET /notifications/unread-count	Signed in	Badge number
PATCH /notifications/read-all	Signed in	Mark all read
PATCH /notifications/:id/read	Signed in (own)	Mark one read
GET /dashboard	Signed in	Role-specific dashboard data
GET /admin/doctors	Admin	Doctor queue
GET /admin/doctors/:id	Admin	Doctor detail + stats
PATCH /admin/doctors/:id/status	Admin	Approve / reject / revoke
GET /admin/users	Admin	User list

Method + path	Access	Purpose
PATCH /admin/users/:id/status	Admin	Activate / deactivate
GET /admin/settings, PUT /admin/settings	Admin	Read / update settings
PUT /admin/settings/logo, DELETE /admin/settings/logo	Admin	Upload / remove logo
GET /uploads/branding/<file>	Public (static, not under /api)	Logo image

6.2 Business rules summary

Rule	How it is enforced
No double booking	Slot row lock FOR UPDATE + partial unique index; per-patient advisory lock + overlap check; per-doctor advisory lock for creating slots.
Fee = doctor fee + commission	Calculated only on the server (utils/fees.js) and snapshotted onto the appointment.
No patient cancel/reschedule within 24 h	422 LOCK_WINDOW; the actions flags hide the buttons.
Confirmation before start - 24 h	Otherwise auto-moved (>= 48 h ahead), max 2 times, then cancelled.
Booking inside 24 h	Confirmed immediately; cannot be changed.
Doctor cancels	Reason required; slot becomes blocked.
Complete / no-show / prescription	Only own doctor, only after the start, only for the right statuses.
Privacy	Others' appointments return 404; doctor notes hidden from patients; certificates private; password hash never sent.
Pending doctors	Can sign in, cannot publish slots, are not listed, cannot receive bookings.
Deactivation / revocation	Future appointments cancelled and the other side notified; deactivated users rejected on the next request.
Times	Stored as timestamptz (UTC); shown in each viewer's local zone; slots generated in the doctor's browser zone; notification texts use DISPLAY_TIMEZONE.

6.3 Security checklist

- bcrypt password hashing; timing-equal login; same error for unknown e-mail and wrong password.
- Signed JWT with a secret from .env; 7-day expiry; user reloaded from the DB on every request.
- Role checks on the server for every protected route; 404 instead of 403 for other people's records.
- zod validation that strips unknown fields; body size limit 200 KB.
- Rate limiting on authentication routes.
- helmet security headers; CORS allow-list; X-Powered-By disabled.
- Upload allow-lists, size limits, magic-number checks, random file names, no SVG, private certificates, cleanup of failed uploads, path.basename against path tricks.
- 5xx details only in the server log; generic message to users.
- Secrets (.env) and uploads excluded from git.

6.4 Commands you will use

Command (folder)	What it does
start.bat (root)	Start Postgres, install if needed, set up on first run, open API + app
stop.bat (root)	Stop the portable Postgres
npm run dev (server)	API with auto-restart (nodemon)
npm start (server)	API without auto-restart
npm run db:setup (server)	Create the database, run migrations, seed admin + demo data
npm run db:reset (server)	Drop all tables, recreate, re-seed (deletes all data!)

Command (folder)	What it does
npm run db:seed / db:seed:demo (server)	Seed admin + settings / plus demo data
npm run test:e2e (server)	Run the 17 end-to-end checks (API must be running)
npm run dev (client)	Vite dev server on 5173
npm run build / preview (client)	Production build into dist/ / serve it locally
pg_ctl -D %USERPROFILE%\pgsql\data start | stop	Start / stop Postgres by hand
initdb -D ... -U postgres --pwprompt	One-time creation of the Postgres data folder

6.5 Demo accounts

Role	E-mail	Password
Patient	patient@demo.local	Demo@1234
Doctor (approved)	ananya.rao@demo.local, rahul.menon@demo.local, fatima.sheikh@demo.local	Demo@1234
Doctor (pending)	vikram.iyer@demo.local	Demo@1234
Admin	admin@docbook.local	Admin@1234

6.6 Common problems

Symptom	Cause and fix
Vite: "http proxy error ... ECONNREFUSED"	The backend (port 5000) is not running. Start it (npm run dev in server, or start.bat).
Backend: "Could not connect to PostgreSQL"	Postgres is not running or DB_* in .env is wrong. Start Postgres (pg_ctl ... start) and check DB_PASSWORD.
pg_ctl: directory ".../data" does not exist	initdb was never run. Run initdb once.
database "doctor_appointments" does not exist	Run npm run db:setup in server.
"Your session has expired"	The token is older than 7 days, or JWT_SECRET changed. Sign in again.
429 Too many attempts	More than 30 auth requests in 15 minutes from one IP. Wait.
PowerShell: "Unexpected token -D"	cmd syntax used in PowerShell. Use & "$env:USERPROFILE\..." instead of "%USERPROFILE%\...".

Part 7 - Glossary

Term	Meaning
API	Application Programming Interface - here, the set of URLs the frontend calls on the backend.
Advisory lock	A named lock in Postgres used to make operations for one patient/doctor happen one at a time.
Association	A relationship between models (hasOne, hasMany, belongsTo).
Bcrypt	Slow, salted password hashing algorithm.
Bearer token	A token sent in the Authorization header: "Bearer <jwt>".
Blob / object URL	Binary file data in the browser and a temporary URL pointing to it.
CommonJS / ES modules	Two module systems: require/module.exports vs import/export.
Commit / rollback	Make a transaction's changes permanent / undo them all.
Component	A React function that returns UI.
CORS	Browser rule controlling which websites may call an API from another origin.
Cron	A schedule format; "* * * * *" = every minute.
Debounce	Waiting until typing pauses (300 ms) before searching.
Endpoint / route	One method + path the API answers, e.g. POST /api/appointments.
ENUM	A database column limited to a fixed list of values.
Foreign key	A column referencing another table's primary key.
Hash	A one-way fingerprint of data.
HMAC-SHA256 (HS256)	Signing method for the JWT using a shared secret.
Idempotent	Doing it twice has the same effect as once (e.g. confirm).
Index	Database structure that speeds up lookups; a unique index also forbids duplicates.
Interceptor	axios hook run on every request/response.
JSON / JSONB	Text data format / Postgres binary JSON column.
JWT	JSON Web Token - signed token proving who you are.
localStorage	Browser storage that survives restarts; holds the token and theme.
Magic number	The first bytes of a file that reveal its real type.
Middleware	A function in Express's request chain.
Migration	A versioned script that changes the database structure.
Model	A Sequelize class representing a table.
Multipart/form-data	Request format that can carry files.
ORM	Object-Relational Mapper (Sequelize) - use tables as JavaScript objects.
Pagination	Splitting long lists into pages (page, limit, total, totalPages).
Primary key / UUID	A row's unique id / a random 128-bit id.
Proxy (Vite)	Forwards /api from port 5173 to 5000 in development.
Rate limiting	Capping requests per time window.
Salt	Random data mixed into a password before hashing.
Scope (Sequelize)	A saved query option, e.g. hide or include passwordHash.
Seed	Inserting initial/demo data.
SKIP LOCKED	Postgres option: ignore rows another transaction has locked.
Snapshot (price)	Copying the fee onto the appointment so later changes do not affect it.
SPA	Single-page application - one HTML page, React swaps the content.
State / context	Data a React component remembers / data shared across components.

Term	Meaning
timestamptz / UTC	Postgres time with zone / universal reference time.
Toast	Small temporary pop-up message.
Transaction	Group of database changes that succeed or fail together.
Validation (zod)	Checking and cleaning input against a schema.